

CLASE 2 · PROGRAMACIÓN II · 2026

CLASE 2

Programación Orientada a Objetos (POO)

3 hs · 2 bloques · actividad práctica incluida



¿Qué hacemos en esta clase?

BLOQUE 1 — 80 min

Las bases de POO

5 min	Conexión con lo que sabés
10 min	¿Por qué POO?
10 min	Clase, objeto, instancia
15 min	Ámbito y modificadores de acceso
20 min	Encapsulamiento y propiedades
20 min	Constructor y sobrecarga



DESCANSO — 15 min

BLOQUE 2 — 80 min

Relaciones y contratos

15 min	Composición y herencia
20 min	virtual / base / override
10 min	Polimorfismo
15 min	Interfaces y clases abstractas
20 min	Actividad práctica

Meta: que al final puedas crear, instanciar y relacionar tus propias clases en C#

Ya sabés programar.

Ahora vas a programar de otra manera.

LO QUE YA TENÉS

- ✓ Variables y tipos de dato
- ✓ if / else / switch
- ✓ for / foreach / while
- ✓ Métodos y funciones
- ✓ Listas y arrays
- ✓ Ordenamiento y búsqueda



LO QUE VAS A APRENDER

- ◆ Clases y objetos
- ◆ Encapsulamiento
- ◆ Herencia
- ◆ Polimorfismo
- ◆ Interfaces
- ◆ Relaciones entre entidades

El problema que la POO resuelve

El mismo sistema, dos formas de pensar

✗ Sin POO (lo que conocés)

```
string nom1 = "Ana";
int    edad1 = 20;
string carr1 = "Sistemas";

string nom2 = "Luis";
int    edad2 = 22;
string carr2 = "Diseño";

// Para imprimir, pasás todo
void Mostrar(string n,
    int e, string c) { ... }
Mostrar(nom1, edad1, carr1);
Mostrar(nom2, edad2, carr2);
```

✓ Con POO (lo que vas a aprender)

```
class Alumno {
    string Nombre;
    int    Edad;
    string Carrera;

    void Mostrar() {
        Console.WriteLine(
            $"{Nombre}...");
    }
}

var a1 = new Alumno();
a1.Mostrar();
```

La POO agrupa datos + comportamiento en una sola unidad. El código refleja el entorno real.

Clase vs. Objeto

El molde no es lo mismo que lo que fabricás con él

CLASE

El molde / la plantilla

- Define qué datos tiene (atributos)
- Define qué puede hacer (métodos)
- No ocupa memoria por sí sola
- Es única — se define una vez

```
class Alumno { ... }
```

OBJETO

La instancia / el ejemplar

- Creado a partir de una clase
- Tiene sus propios valores
- Ocupa memoria (heap)
- Podés tener miles de objetos

```
var a = new Alumno();
```

Anatomía de una clase en C#

Cada parte tiene un rol — no mezclamos responsabilidades

```
public class Alumno {  
  
    // ① CAMPOS (privados)  
    private string _nombre;  
    private int    _edad;  
  
    // ② CONSTRUCTOR  
    public Alumno(string nom, int ed) {  
        _nombre = nom;  
        _edad   = ed;  
    }  
  
    // ③ PROPIEDADES  
    public string Nombre => _nombre;  
  
    // ④ MÉTODOS  
    public void Mostrar() { ... }  
}
```

①

Campos

Datos internos. Siempre private. El entorno exterior no los toca directamente.

②

Constructor

Se llama con new. Inicializa el objeto. Es obligatorio si querés datos al nacer.

③

Propiedades

La ventana al estado interno. Controlás qué se lee y qué se escribe.

④

Métodos

El comportamiento. Lo que el objeto sabe hacer.

¿Quién puede ver qué?

El ámbito y los modificadores de acceso son la base del encapsulamiento

⚠ El problema: todo público

```
class CuentaBancaria {  
    public string Titular;  
    public decimal Saldo;  
    public string Clave;  
}  
  
var c = new CuentaBancaria();  
c.Saldo = -99999; // 🤖 válido  
c.Clave = "1234"; // 🤖 válido
```

Si todo es público, cualquier parte del programa puede romper las reglas de negocio. No hay ningún control.

MODIFICADORES DE ACCESO

Modificador	¿Quién lo ve?
public	Cualquiera — sin restricción
private	Solo dentro de la clase
protected	La clase y sus subclases
internal	Dentro del mismo proyecto

Regla: los campos siempre `private`. Si el entorno exterior necesita acceso, usás propiedades. Esto es exactamente el encapsulamiento (próxima filmína).

Ámbito: local (en el método) → instancia (en la clase) → estático (compartido por todos los objetos)

Encapsulamiento

Los datos viven adentro. El entorno ve solo lo que vos decidís.

❌ Campo público (no hagas esto)

```
public class Alumno {  
    // ⚠️ cualquiera modifica  
    public int Edad;  
}
```

```
a.Edad = -50; // válido 😬
```

✅ Propiedad con validación

```
public class Alumno {  
    private int _edad;  
    public int Edad {  
        get => _edad;  
        set => _edad =  
            value > 0 ? value : 0;  
    }  
}
```

ATAJO: Auto-property — cuando no necesitás validar

```
// C# lo hace por vos: campo privado + getter + setter en una línea  
public string Nombre { get; set; } // lectura y escritura  
public string Carrera { get; private set; } // solo se escribe desde adentro  
public int Legajo { get; } // solo lectura (init en constructor)  
public DateTime Alta { get; } = DateTime.Now; // valor por defecto
```


Constructor e instanciación

Un objeto puede nacer de distintas formas, la sobrecarga lo hace flexible

```
public class Producto {  
    public string Nombre { get; }  
    public decimal Precio { get; private set; }  
    public int Stock { get; private set; }  
    // Constructor principal (el 'real')  
    public Producto(string nom,  
        decimal precio, int stock) {  
        Nombre=nom; Precio=precio;  
        Stock=stock;  
    }  
    // Sobrecarga 1: sin stock → usa 0  
    public Producto(string nom, decimal precio)  
        : this(nom, precio, 0) { }  
    // Sobrecarga 2: solo nombre  
    public Producto(string nom)  
        : this(nom, 0m, 0) { }  
}
```

¿Cuándo se llama?

Al escribir `new Producto(...)`. Es el primer código que ejecuta el objeto.

¿Qué es : `this(...)`?

Llama a otro constructor de la misma clase. Evitás repetir código de inicialización.

¿Cuántos podés tener?

Los que necesites. C# elige el correcto según los parámetros que pasás.

Las tres formas de instanciar:

```
new Producto("Café", 2500m, 100);  
new Producto("Té", 1200m);  
new Producto("Sin definir");
```

Las clases no viven solas

Dos formas de relacionar clases: composición y herencia

COMPOSICIÓN

"TIENE UN"

Una clase usa otra como atributo.

```
class Direccion {  
    public string Calle;  
}  
  
class Alumno {  
    // Alumno TIENE UNA Dirección  
    public Direccion Dom;  
}
```

Alumno → Dirección

HERENCIA

"ES UN"

Una clase extiende otra más general.

```
class Persona {  
    public string Nombre;  
}  
  
// Alumno ES UNA Persona  
class Alumno : Persona {  
    public int Legajo;  
}
```

Alumno → Persona (es más específico)

Herencia

Reutilizás código. Pero si abusás, lo complicás.

```
class Persona {  
    public string Nombre { get; set; }  
    public int Edad { get; set; }  
  
    public virtual string Presentarse() =>  
        $"Hola, soy {Nombre}";  
}  
  
class Alumno : Persona {  
    public int Legajo { get; set; }  
  
    public Alumno(string nom, int leg)  
        : base(nom, leg) { }  
  
    public override string Presentarse() =>  
        $"Alumno {Nombre}, leg. {Legajo}";  
}
```

⚡ Preferí composición cuando la relación no es claramente "es-un"

base()

Llama al constructor del padre. Siempre se ejecuta antes que el hijo.

virtual

Marca un método como redefinible. Sin virtual, no podés hacer override.

override

Redefine el comportamiento en la subclase. Reemplaza la versión del padre.

: Padre

Sintaxis de herencia en C#. Solo se puede heredar de UNA clase.

virtual · override · base

El error más común en herencia — y cómo evitarlo

❌ SIN virtual — method hiding

```
class Animal {
    public string Hablar() => "...";
}
class Perro : Animal {
    // 'new' oculta, no reemplaza
    public new string Hablar() => "Guau";
}
Animal a = new Perro();
a.Hablar(); // "..." 🤖 Animal gana
```

base — accedé al padre desde la subclase

```
public override string Hablar() {
    // llama al método del padre primero
    return base.Hablar() + " ¡Guau!";
} // resultado: "... ¡Guau!"
```

✅ CON virtual + override — polimorfismo real

```
class Animal {
    public virtual string Hablar() => "...";
}
class Perro : Animal {
    // override reemplaza de verdad
    public override string Hablar() => "Guau";
}
Animal a = new Perro();
a.Hablar(); // "Guau" ✅ Perro gana
```

- 1 virtual va en la clase BASE — habilita la sobreescritura
- 2 override va en la SUBCLASE — confirma que la reemplaza
- 3 base llama al padre: en constructor (: base()) o en método (base.X())
- 5 sealed override impide que una subclase vuelva a redefinir ese método

Polimorfismo

El mismo mensaje, comportamientos distintos según quién lo recibe

```
class Figura {  
    public virtual double Area() => 0;  
}  
  
class Circulo : Figura {  
    public double Radio { get; set; }  
    public override double Area() =>  
        Math.PI * Radio * Radio;  
}  
  
class Rectangulo : Figura {  
    public double Base, Alto;  
    public override double Area() =>  
        Base * Alto;  
}
```

La magia:

```
var fs = new List<Figura> {  
    new Circulo(),  
    new Rectangulo(),  
};  
foreach (var f in fs)  
    Console.WriteLine(f.Area());
```

¿Qué hace el foreach?

- Llama a Area() en cada figura
- Cada una ejecuta SU versión
- No importa el tipo exacto
- El código que llama no cambia



DESCANSO

15 minutos

Sistema Escolar

Consigna — paso a paso

- 1 Creá la clase Persona: Nombre (get), Email (get), constructor, y el método virtual Presentarse().
- 2 Creá Alumno que herede de Persona: Legajo (int), lista de materias (List<string>), constructor, override de Presentarse(), y método Inscribirse(string materia).
- 3 Creá Docente que herede de Persona: Nro de empleado (string), lista de asignaturas, constructor, override de Presentarse(), y método AsignarMateria(string).
- 4 Creá la clase Materia: Codigo, Nombre, Cupo (int), List<Alumno> Inscriptos. Método Inscribir() que verifique que haya cupo antes de agregar.
- 5 En Program.cs: creá 3 alumnos y 2 docentes. Cargalos en una List<Persona> y usá foreach para llamar Presentarse() en todos.
- 6 Creá una Materia con cupo 2, inscribí 2 alumnos y verificá qué pasa al intentar inscribir un tercero.
- 7 **BONUS:** Armá un menú que permita ingresar por consola todos los datos y mostrar de manera visual e interactiva el sistema.

Diagrama de clases

Persona

└─ Alumno : Persona

└─ Docente : Persona

Materia (composición con Alumno)

Para tener en cuenta

- En el metodo Presentarse() modifica el saludo segun los datos de la subclase
- List<Persona> acepta Alumno y Docente
- Inscribir() debe validar cupo con if
- ObtenerPromedio() → suma/count notas
- Usá base(Nombre, Email) en constructor de subclases